

1. 设置全局配置

```
In [ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

TARGET_MODEL_PATH = "./resnet20_cifar10-0597-9b0024ac.pth" # 预训练的目标模型路径

BATCH_SIZE = 128 # 每个批次的样本数量
NUM_WORKERS = 4 # 数据加载的并行进程数

SURROGATE_EPOCHS = 10 # 代理网络训练的轮数
LEARNING_RATE = 0.01 # 代理网络的初始学习率

# 白盒 PGD 攻击的超参数:
PGD_EPSILON = 8 / 255.0 # 最大扰动范围 ( $L_\infty$  范数)
PGD_ALPHA = 2 / 255.0 # 每次迭代的步长
PGD_STEPS = 10 # 迭代次数

# 黑盒攻击超参数
BB_PGD_EPSILON = 8 / 255.0
BB_PGD_ALPHA = 2 / 255.0
BB_PGD_STEPS = 10
```

2. Cifar10数据集

- 这一部分加载数据集并且对训练集做简单的数据增强

```
In [2]: transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4), # 随机裁剪, 边缘填充4个像素
    transforms.RandomHorizontalFlip(), # 随机水平翻转
    transforms.ToTensor(), # 将图像转换为张量
])
transform_test = transforms.Compose([
    transforms.ToTensor(), # 将图像转换为张量
])

# 加载 CIFAR-10 训练集与测试集
train_set = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform_train
)

test_set = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform_test
)

# 使用 DataLoader 将数据集封装成可迭代的批次-
train_loader = DataLoader(
    train_set,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS
)

test_loader = DataLoader(
    test_set,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS
)
```

```
print(f"训练集共 {len(train_set)} 张图像，测试集共 {len(test_set)} 张图像。")
```

训练集共 50000 张图像，测试集共 10000 张图像。

3. 设置目标模型

- 从pytorchcv中加载resnet20_cifar10模型结构，并读取下载后对应的权重文件

```
In [3]: from pytorchcv.model_provider import get_model as ptcv_get_model

target_model = ptcv_get_model("resnet20_cifar10", pretrained=False) # 获取 ResNet20_CIFAR10 模型
target_model = target_model.to(device)

checkpoint = torch.load(TARGET_MODEL_PATH, map_location=device) # 加载预训练模型权重
target_model.load_state_dict(checkpoint) # 将模型参数加载到目标模型中

target_model.eval() # 设置模型为评估模式

print("ResNet20_CIFAR10 模型加载并设置为 eval 模式完成。")
```

ResNet20_CIFAR10 模型加载并设置为 eval 模式完成。

4. 定义代理模型

- 使用CNN作为代理网络

```
In [4]: class SurrogateCNN(nn.Module):
    def __init__(self):
        super(SurrogateCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1) # 第一层卷积：输入通道 3，输出通道 32
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1) # 第二层卷积：输入通道 32，输出通道 64
        self.pool = nn.MaxPool2d(2, 2) # 2x2 最大池化层
        self.dropout = nn.Dropout(0.25) # 定义 Dropout 层，丢弃比例为 0.25
        self.fc1 = nn.Linear(64 * 16 * 16, 256) # 第一个全连接层，输入维度 64*16*16，输出维度 256
        self.fc2 = nn.Linear(256, 10) # 第二个全连接层，输入维度 256，输出维度 10
```

```
def forward(self, x):
    x = F.relu(self.conv1(x)) # 第一层卷积 + ReLU 激活函数, 输出形状为 [B, 32, 32, 32], 其中 B 是批次大小
    x = self.pool(F.relu(self.conv2(x))) # 第二层卷积 + ReLU + 池化, 输出形状为 [B, 64, 16, 16]
    x = self.dropout(x) # Dropout, 形状仍为 [B, 64, 16, 16]
    x = x.view(-1, 64 * 16 * 16) # 展平操作, 将张量展平为 [B, 64*16*16]
    x = F.relu(self.fc1(x)) # 第一个全连接层 + ReLU 激活函数
    x = self.dropout(x)
    x = self.fc2(x) # 第二个全连接层
    return x
```

```
surrogate_model = SurrogateCNN().to(device)
```

5. PGD攻击函数

- 图像备份
- 初始化随机扰动
- 迭代计算梯度
- 沿梯度符号方向更新
- 将更新后的图像投影回原始图像的 L_∞ 球

```
In [5]: def pgd_attack(model, images, labels, epsilon, alpha, num_steps):
        """
        在给定模型 model 上, 对输入 images 进行 PGD 攻击, 生成对抗样本并返回。
        输入:
            - model: 要攻击的神经网络 (已设置为 eval 模式)
            - images: 原始输入图像张量
            - labels: 对应的真实标签张量
            - epsilon:  $L_\infty$  最大扰动幅度
            - alpha: 每步迭代的步长
            - num_steps: PGD 迭代次数
        输出:
            - perturbed_images: 被扰动后的对抗样本, 形状同 images
        """
```

```

original_images = images.clone().detach().to(device) # 确保原始图像在计算图中不被修改
perturbed_images = images.clone().detach().to(device) # 复制原始图像作为对抗样本起点
perturbed_images += torch.zeros_like(perturbed_images).uniform_(-epsilon, epsilon) # 在原始图像上添加随机扰动,
perturbed_images = torch.clamp(perturbed_images, 0, 1).detach() # 确保扰动后的图像仍在 [0, 1] 范围内

for _ in range(num_steps): # 迭代 num_steps 次进行攻击
    perturbed_images.requires_grad = True # 允许对抗样本计算梯度

    outputs = model(perturbed_images) # 前向传播, 获取模型输出
    loss = F.cross_entropy(outputs, labels) # 计算交叉熵损失

    model.zero_grad() # 清除模型的梯度缓存
    loss.backward() # 反向传播, 计算损失对 perturbed_images 的梯度
    grad = perturbed_images.grad.data # 获取梯度数据

    perturbed_images = perturbed_images.detach() + alpha * torch.sign(grad) # 沿梯度符号方向更新对抗样本

    delta = torch.clamp(perturbed_images - original_images, min=-epsilon, max=epsilon) # 限制扰动范围
    perturbed_images = torch.clamp(original_images + delta, 0, 1).detach() # 确保扰动后的图像仍在 [0, 1] 范围内

return perturbed_images # 返回最终的对抗样本

```

6. 白盒PGD测试函数

- 遍历测试机, 对每个批次生成对抗样本并让目标模型进行推理, 计算目标模型在PGD对抗样本上的准确率

```

In [6]: def test_whitebox_pgd(model, dataloader, epsilon, alpha, steps):
        """
        在测试集上对 model 进行白盒 PGD 测试, 计算其在对抗样本上的准确率。
        输入:
            - model: 已加载好权重的目标模型
            - dataloader: 测试集的 DataLoader
            - epsilon: PGD 的最大扰动幅度
            - alpha: 每步迭代的步长
            - steps: PGD 迭代次数
        输出:

```

```

- acc: 在对抗样本上的准确率
"""
model.eval() # 确保模型在评估模式
total = 0 # 用于累计测试样本总数
correct = 0 # 用于累计对抗样本被正确分类的数量

for images, labels in dataloader: # 遍历测试集
    images, labels = images.to(device), labels.to(device)

    adv_images = pgd_attack(model, images, labels, epsilon, alpha, steps) # 使用 PGD 生成对抗样本

    with torch.no_grad(): # 关闭梯度计算
        outputs = model(adv_images) # 对对抗样本做前向推理
        preds = outputs.max(1)[1] # 取每行 logits 中概率最大的索引作为预测
        correct += (preds == labels).sum().item() # 累加预测正确的数量
        total += labels.size(0) # 累加总样本数

acc = correct / total # 计算准确
print(f"[White-Box PGD]  $\epsilon$ ={epsilon:.4f},  $\alpha$ ={alpha:.4f}, steps={steps}, 准确率 = {correct}/{total} = {acc:.4f}")
return acc

```

7. 训练代理模型函数

```

In [7]: def train_surrogate(model, train_loader, num_epochs, lr):

    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_decay=5e-4) # 使用 SGD 优化器
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1) # 使用 StepLR 调度器

    for epoch in range(num_epochs):
        model.train() # 确保模型在训练模式
        running_loss = 0.0 # 用于累计每步的损失
        total = 0 # 用于累计总样本数
        correct = 0 # 用于累计预测正确的数量

        for i, (images, labels) in enumerate(train_loader): # 遍历训练集
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad() # 梯度清零

```

```

outputs = model(images) # 前向传播
loss = F.cross_entropy(outputs, labels) # 计算交叉熵损失
loss.backward() # 反向传播, 计算梯度
optimizer.step() # 优化器更新参数

running_loss += loss.item() * labels.size(0) # 累积 loss 和准确率
_, predicted = outputs.max(1) # 获取预测类别
total += labels.size(0)
correct += (predicted == labels).sum().item()

scheduler.step() # 更新学习率 (如果到达预设 step)

epoch_loss = running_loss / total # 平均损失
epoch_acc = correct / total # 平均准确率
print(f"代理模型训练 第 {epoch+1}/{num_epochs} 轮  Loss: {epoch_loss:.4f}  Acc: {epoch_acc:.4f}")

```

8. 黑盒迁移攻击测试函数

- 现在代理模型上生成对抗样本, 再将这些对抗样本投入目标模型, 计算其在目标模型上的准确率

```

In [8]: def test_blackbox_transfer(target_model, surrogate_model, dataloader, epsilon, alpha, steps):

    surrogate_model.eval() # 代理模型评估模式
    target_model.eval() # 目标模型评估模式

    total = 0 # 用于统计测试集总样本数
    correct_on_target = 0 # 用于统计目标模型在对抗样本上的正确数

    for images, labels in dataloader: # 遍历测试集
        images, labels = images.to(device), labels.to(device)

        adv_on_surrogate = pgd_attack(surrogate_model, images, labels, epsilon, alpha, steps) # 在代理模型上生成对

        with torch.no_grad(): # 关闭梯度计算
            outputs = target_model(adv_on_surrogate) # 在目标模型上进行前向推理
            preds = outputs.max(1)[1] # 取每行 logits 中概率最大的索引作为预测

```

```

        correct_on_target += (preds == labels).sum().item() # 累加目标模型预测正确的数量
    total += labels.size(0) # 累加总样本数

transfer_acc = correct_on_target / total # 计算迁移准确率
print(f"[Black-Box via Surrogate]  $\epsilon$ ={epsilon:.4f},  $\alpha$ ={alpha:.4f}, steps={steps}, "
      f"在目标模型上的迁移准确率 = {correct_on_target}/{total} = {transfer_acc:.4f}")
return transfer_acc

```

9. 计算单一类别准确率函数

- 计算白盒PGD对抗后目标模型在每个类别上的准确率

```

In [9]: def per_class_acc_whitebox(model, dataloader, epsilon, alpha, steps, num_classes=10):

    model.eval() # 模型在评估模式
    total_per_class = [0] * num_classes # 初始化每个类别的总样本数
    correct_per_class = [0] * num_classes # 初始化每个类别的正确预测数

    for images, labels in dataloader: # 遍历数据加载器中的批次
        images, labels = images.to(device), labels.to(device)
        adv_images = pgd_attack(model, images, labels, epsilon, alpha, steps) # 生成对抗样本

        with torch.no_grad(): # 关闭梯度计算
            outputs = model(adv_images) # 在对抗样本上进行前向推理
            preds = outputs.max(1)[1] # 取每行 logits 中概率最大的索引作为预测

            for true_label, pred_label in zip(labels, preds): # 遍历每个样本的真实标签和预测标签
                cls = true_label.item() # 获取真实标签的类别索引
                total_per_class[cls] += 1 # 累加该类别的总样本数
                if pred_label.item() == cls: # 如果预测正确，累加正确预测数
                    correct_per_class[cls] += 1

    per_count = list(zip(correct_per_class, total_per_class)) # 将正确预测数和总样本数打包成元组列表
    per_acc = [
        correct_per_class[i] / total_per_class[i] if total_per_class[i] > 0 else 0.0
        for i in range(num_classes)
    ]

```



```
]
return per_count, per_acc
```

10. 整体流程

- 验证目标模型在干净测试集上的准确率
- 对目标模型进行白盒PGD攻击，打印对抗准确率
- 训练代理模型
- 验证代理模型在干净测试集上的准确率
- 使用代理模型生成对抗样本对目标模型进行黑盒攻击

```
In [10]: def main():

# 验证目标模型在干净测试集上的准确率
target_model.eval() # 设置目标模型为评估模式
correct_nat = 0 # 用于统计干净数据的正确预测数
total_nat = 0 # 用于统计干净数据的总样本数
with torch.no_grad(): # 关闭梯度计算
    for images, labels in test_loader: # 遍历测试集
        images, labels = images.to(device), labels.to(device)
        outputs = target_model(images) # 在干净图像上进行前向推理
        preds = outputs.max(1)[1] # 取每行 logits 中概率最大的索引作为预测
        correct_nat += (preds == labels).sum().item() # 累加预测正确的数量
        total_nat += labels.size(0) # 累加总样本数

nat_acc = correct_nat / total_nat
print(f"[目标模型干净数据准确率] = {correct_nat}/{total_nat} = {nat_acc:.4f}\n")

# 白盒 PGD 攻击测试
print("--- 开始白盒 PGD 攻击测试 ---")
wb_acc = test_whitebox_pgd(
    model=target_model, # 目标模型
    dataloader=test_loader, # 测试集 DataLoader
    epsilon=PGD_EPSILON, # PGD 攻击的最大扰动幅度
    alpha=PGD_ALPHA, # 每步迭代的步长
```

```

        steps=PGD_STEPS, # PGD 迭代次数
    )

# 白盒对抗样本的每类准确率统计
print("\n--- 白盒对抗样本每类准确率统计 ---")
per_count_white, per_acc_white = per_class_acc_whitebox(
    model=target_model, # 目标模型
    dataloader=test_loader,
    epsilon=PGD_EPSILON,
    alpha=PGD_ALPHA,
    steps=PGD_STEPS,
    num_classes=10 # CIFAR-10 的 10 个类别
)
print("\n--- 白盒对抗后各类别准确率 ---")

# CIFAR-10 类别名称列表
class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]
for idx, (correct_c, total_c) in enumerate(per_count_white): # 遍历每个类别的正确预测数和总样本数
    acc_c = per_acc_white[idx]
    print(f"类别 {idx} ({class_names[idx]}): {correct_c}/{total_c} = {acc_c:.4f}")

# 代理模型训练
print("\n--- 开始训练代理模型 ---")
train_surrogate(
    model=surrogate_model, # 代理模型
    train_loader=train_loader,
    num_epochs=SURROGATE_EPOCHS,
    lr=LEARNING_RATE,
)

# 代理模型在干净测试集上的准确率
surrogate_model.eval() # 设置代理模型为评估模式
correct_surr = 0 # 用于统计代理模型在干净数据上的正确预测数
total_surr = 0 # 用于统计代理模型在干净数据上的总样本数
with torch.no_grad():

```

```

        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = surrogate_model(images) # 在干净图像上进行前向推理
            preds = outputs.max(1)[1] # 取每行 logits 中概率最大的索引作为预测
            correct_surr += (preds == labels).sum().item() # 累加预测正确的数量
            total_surr += labels.size(0) # 累加总样本数
    print(f"\n[代理模型干净数据准确率] = {correct_surr}/{total_surr} = {correct_surr/total_surr:.4f}\n")

# 黑盒迁移攻击测试
print("--- 开始黑盒迁移攻击测试 ---")
bb_acc = test_blackbox_transfer(
    target_model=target_model,
    surrogate_model=surrogate_model,
    dataloader=test_loader,
    epsilon=BB_PGD_EPSILON,
    alpha=BB_PGD_ALPHA,
    steps=BB_PGD_STEPS,
)

if __name__ == "__main__":
    main()

```

[目标模型干净数据准确率] = $8914/10000 = 0.8914$

--- 开始白盒 PGD 攻击测试 ---

[White-Box PGD] $\epsilon=0.0314$, $\alpha=0.0078$, steps=10, 准确率 = $2/10000 = 0.0002$

--- 白盒对抗样本每类准确率统计 ---

--- 白盒对抗后各类别准确率 ---

类别 0 (airplane): $0/1000 = 0.0000$

类别 1 (automobile): $0/1000 = 0.0000$

类别 2 (bird): $0/1000 = 0.0000$

类别 3 (cat): $1/1000 = 0.0010$

类别 4 (deer): $2/1000 = 0.0020$

类别 5 (dog): $0/1000 = 0.0000$

类别 6 (frog): $0/1000 = 0.0000$

类别 7 (horse): $0/1000 = 0.0000$

类别 8 (ship): $0/1000 = 0.0000$

类别 9 (truck): $0/1000 = 0.0000$

--- 开始训练代理模型 ---

代理模型训练 第 1/10 轮 Loss: 1.9466 Acc: 0.2879

代理模型训练 第 2/10 轮 Loss: 1.5954 Acc: 0.4198

代理模型训练 第 3/10 轮 Loss: 1.4526 Acc: 0.4733

代理模型训练 第 4/10 轮 Loss: 1.3756 Acc: 0.5023

代理模型训练 第 5/10 轮 Loss: 1.3014 Acc: 0.5315

代理模型训练 第 6/10 轮 Loss: 1.2322 Acc: 0.5571

代理模型训练 第 7/10 轮 Loss: 1.1696 Acc: 0.5790

代理模型训练 第 8/10 轮 Loss: 1.1258 Acc: 0.5979

代理模型训练 第 9/10 轮 Loss: 1.0853 Acc: 0.6149

代理模型训练 第 10/10 轮 Loss: 1.0482 Acc: 0.6294

[代理模型干净数据准确率] = $6859/10000 = 0.6859$

--- 开始黑盒迁移攻击测试 ---

[Black-Box via Surrogate] $\epsilon=0.0314$, $\alpha=0.0078$, steps=10, 在目标模型上的迁移准确率 = $5365/10000 = 0.5365$